

Informe Técnico

Sistema Inteligente de Gestión y Optimización de Rutas de Emergencia

Curso: Programación Avanzada (ICC708): Proyecto Integrador Final

Lenguaje: Python 3.13 (solo librería estándar, sin dependencias externas)

Repositorio: código en la raíz del proyecto; documentación extendida en `docs/`

1. Introducción

Durante un desastre natural, un centro de emergencias recibe cientos de solicitudes que deben priorizarse y atenderse con rutas eficientes. Este proyecto implementa un prototipo que cubre ese flujo completo: registra incidentes en una tabla hash propia, los prioriza con una cola de prioridad basada en heap, genera reportes ordenados con algoritmos de ordenamiento implementados desde cero y calcula rutas óptimas sobre la red vial modelada como grafo, usando BFS, Dijkstra/UCS y A*.

El sistema se ejecuta con tres comandos:

```
python generar_datos.py    # dataset reproducible: 520 incidentes, 55
                             nodos, 115 aristas
python main.py              # escenario integrado (Parte 7)
python experimentos.py      # análisis experimental
```

Todos los resultados citados en este informe provienen de ejecuciones reales reproducibles (generadores con semilla fija) sobre Python 3.13.4 / Windows 11.

2. Diseño de los ADT

El sistema se articula sobre tres tipos de datos abstractos. Sus ubicaciones (`Incident.ubicacion`, `EmergencyCenter.ubicacion`) son identificadores de nodos de la red vial, lo que conecta el modelo de datos con el módulo de rutas:

```

+-----+          ubicacion          +-----+
| Incident | -----> | RoadNetwork |
|-----|          |-----|
| id       |          ubicacion          | adyacencia |
| ubicacion|          +-----> | coordenadas |
| prioridad|          |          +-----+
| tipo     |          |          nodos = centros,
| timestamp|          +-----+      zonas e inter-
| estado (Enum)| | EmergencyCenter |      secciones; peso =
+-----+ |-----|      minutos de viaje
          | id, nombre, |
          | ubicacion   |
          +-----+

```

2.1 Incident

Registro de una solicitud de ayuda. Implementado como `dataclass` con un `Enum` de estados (`PENDIENTE` , `ASIGNADO` , `ATENDIDO` , `CANCELADO`) para que solo existan valores válidos.

Operación	Precondición	Postcondición	Complejidad
<code>prioridad_efectiva(ahora)</code>	<code>ahora</code> es <code>datetime</code>	devuelve <code>prioridad</code> $\times (1 + \text{horas_espera})$, sin efectos	$O(1)$
<code>horas_de_espera(ahora)</code>	ídem	horas desde el reporte, ≥ 0	$O(1)$
<code>desde_fila_csv(fila)</code>	fila con 6 columnas válidas	nuevo <code>Incident</code> equivalente	$O(1)$

Fórmula de prioridad. El enunciado propone `prioridad = severidad  $\times$  factor_tiempo` . Se implementó `prioridad_efectiva = base  $\times$  (1 + horas_de_espera)` :
la severidad (1–5) captura la gravedad intrínseca y el factor temporal crece

linealmente, de modo que ningún incidente sufre inanición: uno leve con dos días de espera termina superando a uno grave recién reportado. El `+1` garantiza que un incidente nuevo valga al menos su severidad.

2.2 EmergencyCenter

Centro de operaciones; `dataclass immutable ( frozen=True )` con `id`, `nombre` y `ubicacion`. No tiene operaciones propias: ancla los orígenes de las búsquedas de rutas.

2.3 RoadNetwork

Grafo no dirigido ponderado con **listas de adyacencia** (`nodo → [(vecino, peso)]`) y coordenadas por nodo para la heurística de A^* . Se eligió lista y no matriz porque la red es dispersa (densidad $\approx 8\%$): la lista usa $O(V + E)$ memoria y entrega los vecinos de un nodo en $O(\text{grado})$, que es el patrón de acceso de BFS/Dijkstra/ A^* .

Operación	Postcondición	Complejidad
<code>agregar_nodo(n, x, y)</code>	nodo presente, sin duplicar	$O(1)$
<code>agregar_arista(o, d, w)</code>	arista en ambas adyacencias (bidireccional)	$O(1)$
<code>vecinos(n)</code>	lista <code>(vecino, peso)</code>	$O(1)$
<code>peso(o, d)</code>	peso de la arista directa o <code>None</code>	$O(\text{grado}(o))$
<code>distancia_euclidiana(a, b)</code>	distancia recta en km o <code>None</code>	$O(1)$
<code>cargar_desde_csv(...)</code>	red completa desde <code>nodos.csv</code> + <code>aristas.csv</code>	$O(V + E)$

Invariantes: toda arista conecta nodos existentes; en modo bidireccional la adyacencia es simétrica; `numero_aristas` cuenta aristas lógicas una vez.

3. Estructuras de datos propias

3.1 Tabla hash (Parte 2)

TablaHash es un mapa clave→valor **sin usar dict** : un arreglo de buckets donde cada bucket es una lista de pares [clave, valor] (encadenamiento separado). Decisiones de diseño:

- **Función de dispersión djb2** ($h = h \cdot 33 + \text{carácter}$, truncada a 32 bits): barata y con buena distribución para claves con prefijo común como I-0001...I-0520.
- **Capacidades primas** (17, 37, 79, ...): reducir módulo un primo dispersa mejor los patrones regulares de claves consecutivas.
- **Redimensionamiento**: cuando el factor de carga supera 0.75 se crece al siguiente primo mayor que el doble y se redispersa todo ($O(n)$ ocasional, $O(1)$ amortizado por inserción).
- Se eligió encadenamiento frente a direccionamiento abierto por la eliminación trivial (sin tombstones) y la degradación suave.

Operación	Promedio	Peor caso
insertar	$O(1)$ amortizado	$O(n)$
buscar / actualizar / eliminar	$O(1)$	$O(n)$
estadísticas	$O(\text{capacidad})$	-

Métricas registradas: factor de carga, colisiones acumuladas en inserciones, buckets usados y tamaño del bucket máximo. Con los 520 incidentes reales: factor 0.382, 307 colisiones, 484/1361 buckets usados, cadena máxima 3.

La tabla se **reutiliza** en dos componentes más: como índice de posiciones de la cola de prioridad y como contador de incidentes por zona en los reportes.

3.2 Heap y cola de prioridad (Parte 3)

Un montículo binario es un árbol completo guardado en un arreglo:

$\text{padre}(i) = (i-1)//2$, hijos en $2i+1$ y $2i+2$. Las reparaciones *floatary* *hundir* recorren un solo camino raíz-hoja: $O(\log n)$.

Se implementaron dos estructuras:

- **MinHeap genérico**: guarda (prioridad, elemento) y compara solo la prioridad. Es la frontera de Dijkstra y A^* , y el auxiliar de `top_k`.

- **PriorityQueue de máximos para incidentes:** la raíz es siempre el incidente más urgente. Mantiene una `TablaHash` `id → índice` actualizada en cada intercambio, lo que habilita `actualizar_prioridad` en $O(\log n)$ (patrón *indexed heap*): localizar cuesta $O(1)$ y reordenar $O(\log n)$.

Operación	Complejidad
<code>insertar(incidente, prioridad)</code>	$O(\log n)$
<code>extraer_mas_urgente()</code>	$O(\log n)$
<code>actualizar_prioridad(id, nueva)</code>	$O(\log n)$
<code>top_k(k)</code> (no destructivo)	$O(n \log n)$

4. Algoritmos

4.1 Ordenamiento (Parte 4)

	MergeSort	QuickSort (3 vías, pivote central)
Peor caso	$O(n \log n)$	$O(n^2)$ (improbable con pivote central)
Promedio	$O(n \log n)$	$O(n \log n)$
Estable	Sí	No en general
Memoria	$O(n)$	$O(n)$ en esta variante

MergeSort divide por la mitad y mezcla mitades ordenadas; su estabilidad se usa en los reportes por timestamp y por frecuencia (los empates conservan el orden original). QuickSort usa pivote central y partición de tres vías (`menores / iguales / mayores`), decisiones que neutralizan los dos peores casos clásicos: listas ya ordenadas/invertidas y claves duplicadas.

Reportes generados (`reportes.py`): incidentes más antiguos (MergeSort por timestamp), más críticos (QuickSort por prioridad efectiva descendente) y zonas con más incidentes (conteo con `TablaHash` + MergeSort por frecuencia).

4.2 Búsqueda en grafos (Parte 6)

Los tres algoritmos devuelven ruta, costo acumulado y nodos visitados (en orden de expansión), como exige el enunciado.

- **BFS:** frontera FIFO; garantiza la ruta con menos tramos ignorando los pesos. $O(V + E)$.
- **Dijkstra / UCS:** expande siempre el nodo de menor costo acumulado usando el `MinHeap` propio con eliminación perezosa (variante estándar sin *decrease-key*). Óptimo en minutos con pesos no negativos. $O((V+E) \log V)$.
- **A*** (bono): ordena la frontera por $f(n) = g(n) + h(n)$ con $h(n) = \text{distancia_euclidiana}(n, \text{destino}) / 60 \text{ km/h}$ expresada en minutos. Ambos comparten el mismo motor: Dijkstra es literalmente A* con $h = 0$.

Admisibilidad de la heurística. Ninguna ruta es más corta que la línea recta (desigualdad triangular) y ninguna vía del dataset supera 55 km/h, menor que los 60 km/h de referencia; el margen absorbe además el redondeo de pesos. La heurística es también consistente, por lo que A* con conjunto de expandidos conserva la optimalidad, verificado empíricamente: en las 8 consultas del experimento A* devolvió exactamente el mismo costo que Dijkstra.

5. Red vial y dataset

`generar_datos.py` produce el dataset completo con semilla fija (`random.Random(20260704)`), garantizando reproducibilidad:

- **Red:** 4 centros fijos en los puntos cardinales + 51 zonas con coordenadas uniformes en un plano de 100×100 km (55 nodos ≥ 50 exigidos). Cada nodo se conecta con sus 3 vecinos más cercanos; un *union-find* garantiza conectividad agregando la arista más corta entre componentes; se añaden atajos aleatorios hasta 115 aristas (≥ 100 exigidas). Peso = distancia / velocidad de la vía (30–55 km/h) en minutos.
- **Incidentes:** 520 (≥ 500 exigidos) con severidad 1–5 en distribución realista (predominio de medias), zonas con pesos de aparición desiguales (para que el reporte de zonas muestre concentración real) y timestamps repartidos en las 72 h previas a la fecha de referencia.

6. Escenario integrado (Parte 7)

Flujo ejecutado por `main.py` con los resultados de la corrida real:

1. **Carga:** 520 incidentes, 4 centros, red de 55 nodos / 115 aristas.

- 2. **Inserción:** todos los incidentes entran a la TablaHash (métricas arriba) y a la PriorityQueue ; se muestra el Top-10 crítico y se demuestra actualizar_prioridad reclasificando I-0100 (47.6 → 95.2).
- 3. **Extracción:** el más urgente es I-0102 , incendio en Z-18 , severidad 5, 71.8 h de espera, prioridad efectiva **364.0**.
- 4. **Centro más cercano** (Dijkstra desde cada centro): C-NORTE 84.6 min, **C-SUR 67.9**, C-ORIENTE 72.0, C-PONIENTE 68.7 → se elige el Centro Sur.
- 5. **Comparación de algoritmos** sobre C-SUR → Z-18 : los tres encuentran la misma ruta de 3 tramos y 67.9 min, pero BFS expande 33 nodos, Dijkstra 23 y A* solo 9.
- 6. **Mantenimiento:** I-0102 pasa a estado asignado (actualizar + verificación con buscar); I-0008 se elimina simulando una cancelación.
- 7. **Reportes:** más antiguos, más críticos y zonas más afectadas (Z-03: 22 incidentes, Z-23: 20, Z-39: 19).

Resumen final: incidente I-0102 asignado al Centro de Emergencia Sur, ruta C-SUR → Z-38 → Z-29 → Z-18 , costo total 67.9 minutos, tiempo estimado 1h 08m.

7. Resultados experimentales

7.1 Hashing

500 búsquedas aleatorias por fila; tiempos en ms:

n	capacidad	carga	colisiones	máx bucket	inserción	búsq. hash	búsq. lineal
100	163	0.613	47	2	0.40	0.76	0.86
500	673	0.743	295	3	2.08	0.72	3.73
1 000	1 361	0.735	750	4	3.72	1.17	9.42
5 000	10 949	0.457	1 742	2	40.31	1.63	41.79
20 000	43 853	0.456	4 968	3	106.11	1.05	219.74

La búsqueda en la tabla es plana (~1 ms) mientras la lineal crece con n hasta ser **200× más lenta**; la cadena máxima nunca pasa de 4 → dispersión uniforme; la inserción total crece linealmente → O(1) amortizado real.

7.2 Heap / cola de prioridad

n	inserción/elem	200 updates	extracción/elem
500	8.62 μs	1.83 ms	36.08 μs
2 000	10.30 μs	2.87 ms	38.09 μs
10 000	11.90 μs	3.99 ms	62.42 μs
50 000	15.46 μs	5.43 ms	84.82 μs

Multiplicar $n \times 100$ solo multiplica el costo unitario $\times 1.8$ –2.4: crecimiento logarítmico ($\log_2 500 \approx 9$ vs $\log_2 50\,000 \approx 15.6$). La extracción es más cara por las dos comparaciones por nivel y el mantenimiento del índice de posiciones; el precio de actualizar prioridades en $O(\log n)$.

7.3 Ordenamiento

Tiempos en ms; `sorted()` (Timsort en C) como referencia:

n	caso	MergeSort	QuickSort	sorted()
1 000	aleatorio	4.91	7.19	0.27
5 000	aleatorio	22.95	16.06	0.53
15 000	aleatorio	73.67	59.77	1.94
15 000	ordenado	45.30	38.67	0.22
15 000	inverso	65.70	38.97	0.25

Ambos escalan $n \log n$ ($\times 15$ en $n \Rightarrow \times 13$ –15 en tiempo, no $\times 225$ como un $O(n^2)$). QuickSort supera a MergeSort en listas grandes y **no degenera** en los casos ordenado/inverso gracias al pivote central + tres vías. La brecha de $\sim 30\times$ frente a `sorted()` mide el costo del intérprete frente a C, no un defecto algorítmico.

7.4 Grafos

Promedio de 8 consultas centro→zona sobre la red real:

Algoritmo	Costo (min)	Nodos visitados	Tiempo
BFS	120.9	36.5	0.041 ms
Dijkstra	108.8	34.1	0.190 ms
A*	108.8	17.9	0.131 ms

Caso ilustrativo C-SUR → Z-48 : BFS eligió 5 tramos con 192.2 min; la ruta óptima usa 6 tramos y cuesta 146.8 min (**31 % menos**): menos tramos no significa menos tiempo. A* iguala siempre el costo de Dijkstra expandiendo en promedio 47 % menos nodos.

8. Discusión

- **La asintótica se observa en la práctica:** búsqueda $O(1)$ plana en la tabla, costos unitarios casi constantes en el heap al centuplicar n , escalado $n \log n$ de ambos ordenamientos.
- **Las constantes también importan:** BFS es $4\times$ más barato por consulta que Dijkstra pero entrega rutas 11 % más caras, inaceptable en emergencias donde el criterio es tiempo de respuesta. `sorted()` gana $30\times$ por estar en C, no por mejor complejidad.
- **Composición de estructuras:** la tabla hash dentro de la cola de prioridad convierte `actualizar_prioridad` de $O(n)$ a $O(\log n)$; el mismo `MinHeap` sirve de frontera a Dijkstra y A*; los reportes combinan tabla y ordenamientos. Cada pieza se usa donde su complejidad es la adecuada.
- **Limitaciones y trabajo futuro:** la red es sintética y estática (sin tráfico dinámico); la asignación es un incidente a la vez sin gestionar flotas ni capacidad de los equipos; `top_k` es $O(n \log n)$ y podría bajarse a $O(n + k \log n)$ con *heapify*. Ninguna limita las conclusiones sobre las estructuras, que son el objeto del curso.

9. Conclusiones

Se construyó un prototipo funcional que integra los siete requerimientos del enunciado con estructuras implementadas desde cero y verificadas: la tabla

hash mantiene $O(1)$ real con factor de carga controlado y cadenas máximas de 3–4 entradas; la cola de prioridad opera en $O(\log n)$ medible incluso con 50 000 elementos; QuickSort y MergeSort escalan como $n \log n$ y sobreviven a sus casos patológicos; y sobre la red vial, Dijkstra y A^* garantizan rutas de tiempo mínimo, con A^* expandiendo la mitad de los nodos gracias a una heurística demostrablemente admisible. El escenario integrado ejecuta el flujo completo, de 520 incidentes en CSV a la asignación del más urgente con ruta, costo y tiempo estimado, en menos de un segundo.

Anexo: reproducción y evidencias

```
python generar_datos.py    # regenera datos/ (semilla fija 20260704)
python main.py             # evidencia de los pasos 1–7 y resumen final
python experimentos.py     # tablas de las secciones 7.1–7.4
```

Documentación extendida por parte en `docs/01...08_*.md`, con diagramas, precondiciones/postcondiciones completas y discusión ampliada.